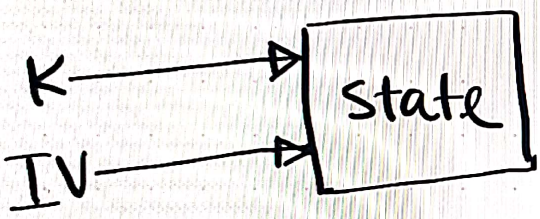
ChaCha 20 : Stream Cipher

Goal: generate deterministic, pseudorandom bits.

The ChaCha20 Keystream generator maintains a state.



The state is initialized with a secret key K and initialization vector IV.

during each iteration,
the Keystream generator generates a block of 64 bytes of Keystream. (512 bits)

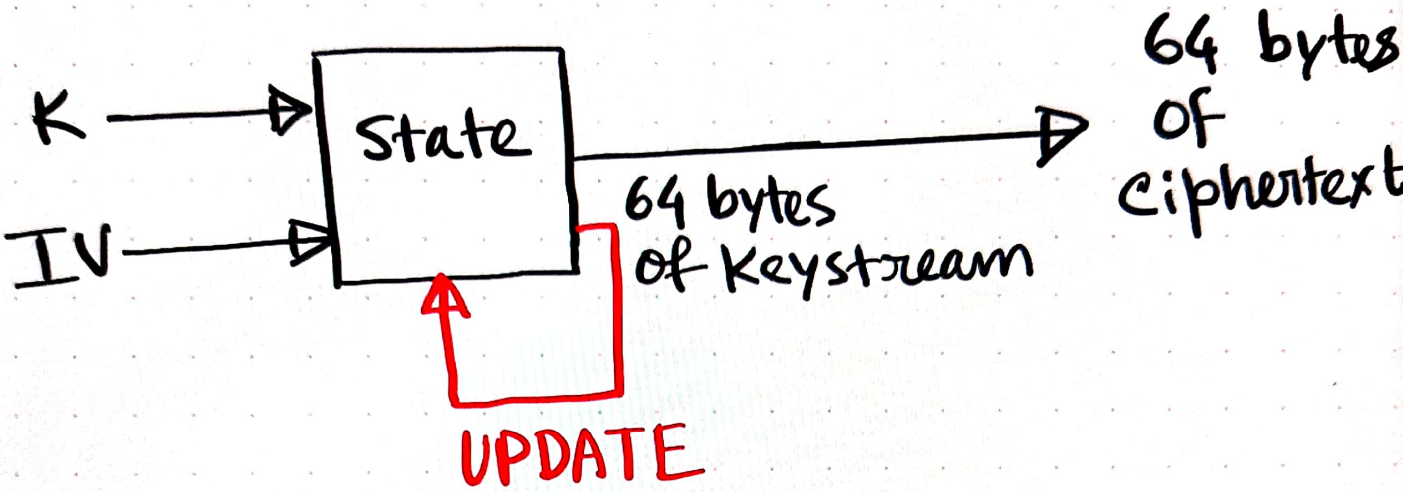
To which a block of 64 bytes (512 bits) of plain text are added.

and then the state is updated.

Scanned with OKEN Scanner

ChaCha20

Keystream Generator



The process is repeated until enough Key Stream bytes have been generated to encrypt the entire plaintext.

ChaCha 20 has 2 components,

(i) An initial State.

(ii) A Quater Round function.

Scanned with OKEN Scanner

Description of the Initial State

Notation :

→ The Secret Key in ChaCha 20, is a randomly selected

256 bit Key $K = (K_1, K_2, K_3, \cdots, K_8)$

Where, K_i is a 32-bit word.
f_i is a 32-bit word.

→ 128 bit constant $f = (f_1, f_2, f_3, f_4)$

- A hexadecimal digit is a 4-bit number:

Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hexadecimal	0	1	2	3	4	5	6	7	8	9	*a*	*b*	*c*	*d*	*e*	*f*

- $f_1 = 0x61707865$, $f_2 = 0x3320646e$, $f_3 = 0x79622d32$, $f_4 = 0x6b206574$.

→ 96-bit nonce $n = (n_1, n_2, n_3)$

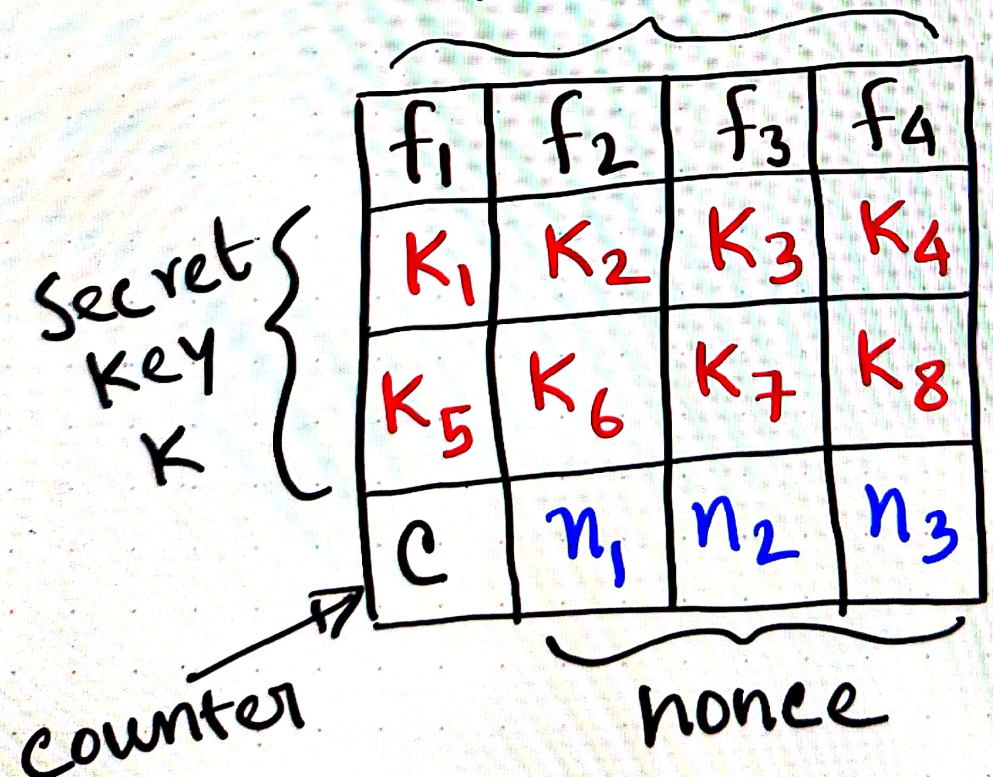
nonce is the initializing value IV
a non-repeating quantity.

Scanned with OKEN Scanner

nonces are usually selected uniformly at random or by incrementing a counter.

→ 32 bit counter C.

The initial state of ChaCha20 can be represented by this MATRIX (4×4) of 32 bit words,



Scanned with OKEN Scanner

f_1	f_2	f_3	f_4
K_1	K_2	K_3	K_4
K_5	K_6	K_7	K_8
C	n_1	n_2	n_3

=

S_1	S_2	S_3	S_4
S_5	S_6	S_7	S_8
S_9	S_{10}	S_{11}	S_{12}
S_{13}	S_{14}	S_{15}	S_{16}

Scanned with OKEN Scanner

The QuaterRound function of ChaCha 20

it's called a quater round function, Since it operates on the 'quater' of the state.

→ On a binary string of len 128 bits

The input to the QRF consists of 4 32-bit words a, b, c, d.

→ The purpose of the QRF is to, mix up the bits of a, b, c, d in a complicated non-linear fashion.

This is done only by,

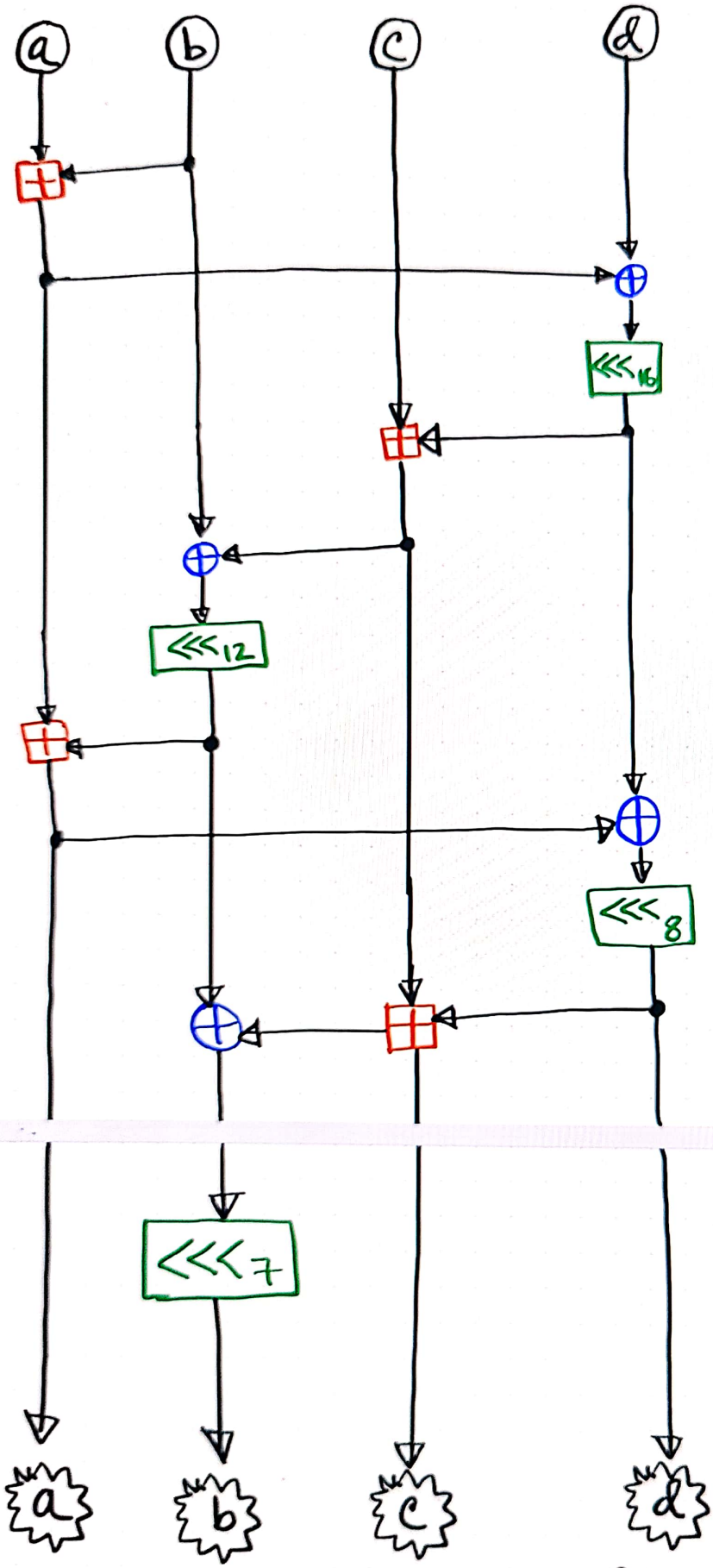
[Box-Sum] : $\boxplus$ int addition modulo 2^{32}

[XOR] : $\oplus$

[Left Rotation] : $\lll t$: left rotation by "t" bit positions.

Scanned with OKEN Scanner

The result yeilded by above operations are all 32-bit words.



all horribly messed up. (diffusion)

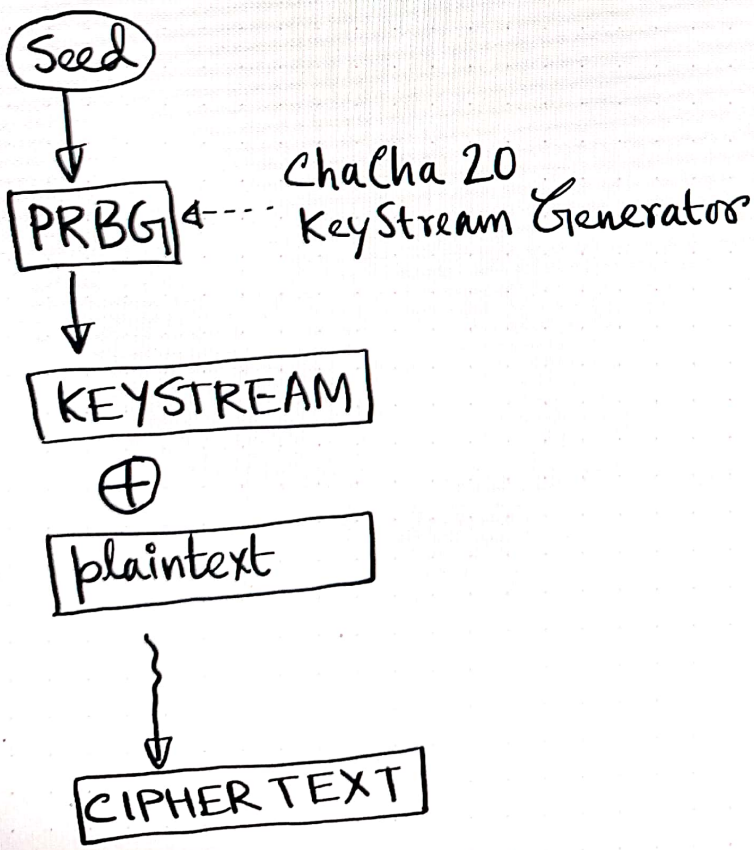
Scanned with OKEN Scanner

```
function QR(a,b,c,d):{
    a ← a ⊞ b ;          a ← a ⊞ b;
    d ← a ⊕ d ;          d ← a ⊕ b;
    d ← d <<<16;         d ← d <<<8

    c ← c ⊞ d ;          c ← c ⊞ d;
    b ← b ⊕ c;           b ← b ⊕ c;
    b ← b <<<12;         b ← b <<<7
    return a,b,c,d;
} end function ;
```


Description of the ChaCha 20

KeyStream Generator



Scanned with OKEN Scanner

The KEYSTREAM GENERATOR of Chacha 20

Suppose, Alice wants to send Bob a message which is 64,000 byte in length.

Each iteration of the key stream generator outputs 64 Keystream bytes.

So, Alice would need to iterate this algorithm a 1000 times to generate all the 64,000 Keystream bytes.

// The counter is used the number of times the main-loop in the KeyStream Generator is used.

So, to encrypt the message m for Bob, Alice does the following,

Scanned with OKEN Scanner

// with random nonce we ensure WHP that Alice never invokes the ChaCha20 KSG with the same inputs.

Select a nonce preferebly at random;

Initialize Counter ← 0 ;

While Keystream bytes are required do:

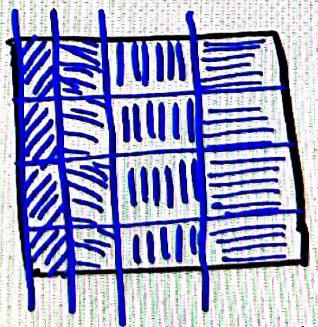
Create the initial state : S ←

f_1	f_2	f_3	f_4
K_1	K_2	K_3	K_4
K_5	K_6	K_7	K_8
C	n_0	n_1	n_2

S' ← S ; // S' is a copy of S.

UPDATE S by doing the following 10 times {

QR(S_1, S_5, S_9, S_{13});
QR(S_2, S_6, S_{10}, S_{14});
QR(S_3, S_7, S_{11}, S_{15});
QR(S_4, S_8, S_{12}, S_{16});



works on the coloumns.

Scanned with OKEN Scanner

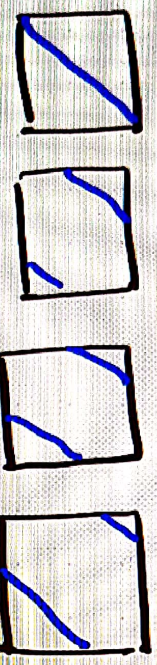
$QR(S_4, S_8, S_{12}, S_{16})$; coloumns.

$QR(S_1, S_6, S_{11}, S_{16})$;

$QR(S_2, S_7, S_{12}, S_{13})$;

$QR(S_3, S_8, S_9, S_{14})$;

$QR(S_4, S_5, S_{10}, S_{15})$;



} Works on diagonals of various type

// at this point the bits are combined in a quite complicated and non-linear way. This we'are doing 10 times!

} return $S \oplus S'$ (64 Keystream bytes 512 bit)

Counter: $C \leftarrow C+1$;

ENCRYPTION: The Keystream bytes are XORed with the plaintext bytes to produce the ciphertext bytes.

The nonce is appended to the Ciphertext.

Scanned with OKEN Scanner

the nonce is appended

DECRYPTION: Upon receiving the nonce appended in the ciphertext, Bob regenerates the Keystream with the same Key and nonce and XORs the Keystream bits with the ciphertext bits.

- Why ChaCha20 is very efficient?
- To generate 64 Keystream bytes, The QR function is invoked 80 times. each invokation requires, 12 arithmetic operations.

So, 64 Keystream bytes can be generated only $12 \times 80 = 960$ many computer instructions.

Scanned with OKEN Scanner

These operations, ARX are also built into the machine at the circuit level So they are all O(1) operations.

Chacha20 incorporates a { nonce and counter } for Keystream generation.

By this it ensures a Keystream is never reused even though the same Sec. Key be used for some period of time.

Since we use a 96 bit nonce for each plaintext that they encrypt and they increment the counter for each 64 byte Keystream block used to encrypt, for A 64 byte block of a particular plaintext, with very high probability, Chacha20 Keystream is never reused by Alice and Bob.

Scanned with OKEN Scanner

for Key Stream generators, we need

(i) INDISTINGUISHABILITY.

(ii) UNPREDICTABILITY.

ChaCha family of Stream Ciphers

One approach of building PRGs is using Blum-Micali paradigm. The other builds PRGs from a more versatile primitive called 'Pseudorandom functions' in Counter mode.

PRGs underlying chacha stream cipher.

They take as input

(i) a 256 bit seed

(ii) a 64 bit nonce.

for now, ignore nonce and set it to 0.

---

reference : A Graduate Course in CRYPTOGRAPHY boneh & Soup.

Scanned with OKEN Scanner

Remarks

→ The design of Chacha is highly parallelizable and can take advantage of multiple processor cores to speed up encryption.

→ it enables random access to the output blocks. The output block j can be computed w/o computing all previous blocks!

Scanned with OKEN Scanner